

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

АВЛ-деревья поиска

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Целью работы является изучение деревьев поиска и получение практических навыков их использования

2 Вариант задания

2.1 Вариант: 11

2.2 Порядок обхода

Симметричный (in-order: левый потомок → корень → правый потомок)

2.3 Задание:

1. Заменить в дереве все отрицательные элементы на их абсолютные величины.
2. Уменьшить в 2 раза все элементы, делящиеся без остатка на 4.
3. Обойти дерево повторно симметричным способом. У каждой обойдённой вершины изменить значение по формуле: $k_i = |k_i - k_{i-1}|$, где i — индекс текущего элемента в порядке обхода. Для первого элемента ($i=0$) предыдущее значение $k_{-1} = 0$.

После каждого этапа дерево перестраивается с сохранением свойства АВЛ-дерева поиска.

3 Листинг программы

```
#include <iostream>
#include <vector>
#include <string>
#include <cmath>
#include <algorithm>
using namespace std;

// Структура узла AVL-дерева
struct Node {
    int value; // хранимое значение
    int height; // высота поддерева с корнем в этом узле
    Node* left; // левый потомок (меньше)
    Node* right; // правый потомок (больше)

    Node(int v) : value(v), height(1), left(nullptr), right(nullptr) {}
};

//
// Вспомогательные функции для балансировки

// Вернуть высоту узла (0, если узел пустой)
int getHeight(Node* n) {
    return n ? n->height : 0;
}

// Пересчитать высоту узла на основе высот потомков
void updateHeight(Node* n) {
    if (n)
        n->height = 1 + max(getHeight(n->left), getHeight(n->right));
}

// Коэффициент баланса: разница высот правого и левого поддеревьев
int bfactor(Node* n) {
    return n ? getHeight(n->right) - getHeight(n->left) : 0;
}

// Малый правый поворот
Node* rotateRight(Node* p) {
    Node* q = p->left;
    p->left = q->right;
```

```

    q->right = p;
    updateHeight(p);
    updateHeight(q);
    return q;
}

// Малый левый поворот
Node* rotateLeft(Node* q) {
    Node* p = q->right;
    q->right = p->left;
    p->left = q;
    updateHeight(q);
    updateHeight(p);
    return p;
}

// Балансировка узла p
Node* balance(Node* p) {
    updateHeight(p);
    int bf = bfactor(p);
    if (bf == 2) {
        if (bfactor(p->right) < 0)
            p->right = rotateRight(p->right); // большой левый поворот
        return rotateLeft(p);
    }
    if (bf == -2) {
        if (bfactor(p->left) > 0)
            p->left = rotateLeft(p->left); // большой правый поворот
        return rotateRight(p);
    }
    return p;
}

// Основные операции AVL-дерева

// Вставить значение k
Node* insert(Node* p, int k) {
    if (!p) return new Node(k);
    if (k < p->value)
        p->left = insert(p->left, k);
    else if (k > p->value)
        p->right = insert(p->right, k);
    return balance(p);
}

```

```
}
```

```
// Найти узел с минимальным значением
```

```
Node* findMin(Node* p) {  
    return p->left ? findMin(p->left) : p;  
}
```

```
// Удалить узел с минимальным значением
```

```
Node* removeMin(Node* p) {  
    if (!p->left) return p->right;  
    p->left = removeMin(p->left);  
    return balance(p);  
}
```

```
// Удалить узел со значением k
```

```
Node* removeNode(Node* p, int k) {  
    if (!p) return nullptr;  
    if (k < p->value) {  
        p->left = removeNode(p->left, k);  
    } else if (k > p->value) {  
        p->right = removeNode(p->right, k);  
    } else {  
        Node* l = p->left;  
        Node* r = p->right;  
        delete p;  
        if (!r) return l;  
        Node* mn = findMin(r);  
        mn->right = removeMin(r);  
        mn->left = l;  
        return balance(mn);  
    }  
    return balance(p);  
}
```

```
// Поиск с подсчётом шагов
```

```
bool search(Node* p, int k, int& steps) {  
    steps = 0;  
    while (p) {  
        steps++;  
        if (k == p->value) return true;  
        else if (k < p->value) p = p->left;  
        else p = p->right;  
    }
```

```

    }
    return false;
}

// Освободить память дерева
void freeTree(Node* p) {
    if (!p) return;
    freeTree(p->left);
    freeTree(p->right);
    delete p;
}

//
// Обходы и вывод
//

// Симметричный обход — заполняет вектор значениями
void inorder(Node* p, vector<int>& result) {
    if (!p) return;
    inorder(p->left, result);
    result.push_back(p->value);
    inorder(p->right, result);
}

// Вывод симметричного обхода в консоль
void printInOrder(Node* p) {
    vector<int> vals;
    inorder(p, vals);
    cout << "In-order traversal: ";
    for (int v : vals) cout << v << " ";
    cout << "\n";
}

// Вывод дерева (повёрнутое на 90°, правая ветвь — сверху)
void printTree(Node* p, int level = 0) {
    if (!p) return;
    printTree(p->right, level + 1);
    cout << string(level * 4, ' ') << p->value << "\n";
    printTree(p->left, level + 1);
}

// Перестройка дерева из вектора

```

```

//
Node* rebuildFromVector(const vector<int>& vals) {
    Node* root = nullptr;
    for (int v : vals)
        root = insert(root, v);
    return root;
}

//
// Задание варианта 11
//

// Шаг 1: abs(отрицательные) + деление на 2 кратных 4
void step1_transform(vector<int>& vals) {
    for (int& v : vals)
        if (v < 0) v = abs(v);
    for (int& v : vals)
        if (v % 4 == 0) v /= 2;
}

// Шаг 2:  $k_i = |k_i - k_{i-1}|$ ,  $k_{-1} = 0$ 
void step2_differenceTransform(vector<int>& vals) {
    int prev = 0;
    for (int& v : vals) {
        int original = v;
        v = abs(v - prev);
        prev = original;
    }
}

//
// Главная функция
//
int main() {
    Node* root = nullptr;
    int choice, val, steps;

    cout << "=== AVL Tree - Variant 11 ===\n";

    while (true) {
        cout << "\n--- Menu ---\n";
        cout << "1. Add element\n";
    }
}

```

```
cout << "2. Delete element\n";
cout << "3. Search element\n";
cout << "4. Print tree (visual)\n";
cout << "5. Print in-order traversal\n";
cout << "6. Execute variant 11 task\n";
cout << "0. Exit\n";
cout << "Your choice: ";
cin >> choice;
```

```
switch (choice) {
```

```
case 1:
```

```
    cout << "Enter value to add: ";
    cin >> val;
    root = insert(root, val);
    cout << "Element " << val << " added.\n";
    break;
```

```
case 2:
```

```
    cout << "Enter value to delete: ";
    cin >> val;
    root = removeNode(root, val);
    cout << "Element " << val << " deleted (if it existed).\n";
    break;
```

```
case 3:
```

```
    cout << "Enter value to search: ";
    cin >> val;
    if (search(root, val, steps))
        cout << "Found! Steps taken: " << steps << "\n";
    else
        cout << "Not found. Steps taken: " << steps << "\n";
    break;
```

```
case 4:
```

```
    cout << "\nTree (right branch on top):\n";
    if (!root) cout << "(empty)\n";
    else printTree(root);
    break;
```

```
case 5:
```

```
    if (!root) cout << "Tree is empty.\n";
```



```

    else printInOrder(root);
    break;

case 6: {
    if (!root) { cout << "Tree is empty!\n"; break; }

    cout << "\n===== Variant 11 task =====\n";

    cout << "\n[0] Initial tree:\n";
    printTree(root);
    printInOrder(root);

    // Mar 1
    vector<int> vals;
    inOrder(root, vals);
    step1_transform(vals);

    cout << "\n[1] After abs(negatives) and halving multiples of 4:\n";
    cout << "    Values: ";
    for (int v : vals) cout << v << " ";
    cout << "\n";

    freeTree(root);
    root = rebuildFromVector(vals);

    cout << "\n    Rebuilt tree after step 1:\n";
    printTree(root);
    printInOrder(root);

    // Mar 2
    vals.clear();
    inOrder(root, vals);

    cout << "\n[2] Applying  $k_i = |k_i - k_{i-1}|$ :\n";
    cout << "    Before: ";
    for (int v : vals) cout << v << " ";
    cout << "\n";

    step2_differenceTransform(vals);

    cout << "    After: ";
    for (int v : vals) cout << v << " ";

```

```

        cout << "\n";

        freeTree(root);
        root = rebuildFromVector(vals);

        cout << "\n[Final] Tree after all transformations:\n";
        printTree(root);
        printInOrder(root);
        cout << "=====\n";
        break;
    }

    case 0:
        freeTree(root);
        cout << "Goodbye!\n";
        return 0;

    default:
        cout << "Invalid choice.\n";
    }
}
return 0;
}

```

3.1 Листинг на Kotlin

```

import kotlin.math.abs
import kotlin.math.max

/*
 * Лабораторная работа №6 — AVL-деревья поиска
 * Вариант 11
 */

//
=====
===
// Структура узла AVL-дерева
//
=====
===

```

```

class Node(
    var value: Int
) {
    var height: Int = 1
    var left: Node? = null
    var right: Node? = null
}

```

```
//
```

```
=====
===
```

```
// Вспомогательные функции
```

```
//
```

```
=====
===
```

```
// Получить высоту узла
```

```
fun getHeight(node: Node?): Int {
    return node?.height ?: 0
}

```

```
// Обновить высоту узла
```

```
fun updateHeight(node: Node?) {
    if (node != null) {
        node.height = 1 + max(
            getHeight(node.left),
            getHeight(node.right)
        )
    }
}

```

```
// Коэффициент баланса
```

```
fun bfactor(node: Node?): Int {
    return if (node != null) {
        getHeight(node.right) - getHeight(node.left)
    } else {
        0
    }
}

```

```
// Правый поворот
```

```
fun rotateRight(p: Node): Node {

```

```

    val q = p.left!!

    p.left = q.right
    q.right = p

    updateHeight(p)
    updateHeight(q)

    return q
}

// Левый поворот
fun rotateLeft(q: Node): Node {
    val p = q.right!!

    q.right = p.left
    p.left = q

    updateHeight(q)
    updateHeight(p)

    return p
}

// Балансировка
fun balance(node: Node): Node {
    updateHeight(node)

    val bf = bfactor(node)

    // Правое поддерево тяжелее
    if (bf == 2) {
        if (bfactor(node.right) < 0) {
            node.right = rotateRight(node.right!!)
        }

        return rotateLeft(node)
    }

    // Левое поддерево тяжелее
    if (bf == -2) {
        if (bfactor(node.left) > 0) {

```

```

        node.left = rotateLeft(node.left!!)
    }

    return rotateRight(node)
}

return node
}

//
=====
===
// Основные операции AVL-дерева
//
=====
===

// Вставка элемента
fun insert(node: Node?, value: Int): Node {
    if (node == null) {
        return Node(value)
    }

    if (value < node.value) {
        node.left = insert(node.left, value)
    } else if (value > node.value) {
        node.right = insert(node.right, value)
    }

    return balance(node)
}

// Найти минимальный элемент
fun findMin(node: Node): Node {
    return node.left?.let { findMin(it) } ?: node
}

// Удалить минимальный элемент
fun removeMin(node: Node): Node? {
    if (node.left == null) {
        return node.right
    }

```

```

        node.left = removeMin(node.left!!)

        return balance(node)
    }

// Удаление элемента
fun removeNode(node: Node?, value: Int): Node? {
    if (node == null) return null

    when {
        value < node.value -> {
            node.left = removeNode(node.left, value)
        }

        value > node.value -> {
            node.right = removeNode(node.right, value)
        }

        else -> {
            val leftChild = node.left
            val rightChild = node.right

            if (rightChild == null) {
                return leftChild
            }

            val minNode = findMin(rightChild)

            minNode.right = removeMin(rightChild)
            minNode.left = leftChild

            return balance(minNode)
        }
    }

    return balance(node)
}

// Поиск элемента
fun search(node: Node?, value: Int): Pair<Boolean, Int> {
    var current = node

```

```

var steps = 0

while (current != null) {
    steps++

    current = when {
        value == current.value -> {
            return Pair(true, steps)
        }

        value < current.value -> {
            current.left
        }

        else -> {
            current.right
        }
    }
}

return Pair(false, steps)
}

//
=====
===
// Обходы дерева
//
=====
===

// Симметричный обход
fun inOrder(node: Node?, result: MutableList<Int>) {
    if (node == null) return

    inOrder(node.left, result)
    result.add(node.value)
    inOrder(node.right, result)
}

// Печать симметричного обхода
fun printInOrder(node: Node?) {

```

```

val values = mutableListOf<Int>()

inOrder(node, values)

print("In-order traversal: ")

for (v in values) {
    print("$v ")
}

println()
}

//
=====
===
// Вывод дерева
//
=====
===
fun printTree(node: Node?, level: Int = 0) {
    if (node == null) return

    printTree(node.right, level + 1)

    println(" ".repeat(level * 4) + node.value)

    printTree(node.left, level + 1)
}

//
=====
===
// Перестройка дерева
//
=====
===
fun rebuildFromVector(values: List<Int>): Node? {
    var root: Node? = null

    for (v in values) {
        root = insert(root, v)
    }
}

```



```

    }

    return root
}

//
=====
===
// Задание варианта 11
//
=====
===

// Шаг 1
fun step1Transform(values: MutableList<Int>) {

    // Отрицательные -> abs
    for (i in values.indices) {
        if (values[i] < 0) {
            values[i] = abs(values[i])
        }
    }

    // Делящиеся на 4 -> /2
    for (i in values.indices) {
        if (values[i] % 4 == 0) {
            values[i] /= 2
        }
    }
}

// Шаг 2
fun step2DifferenceTransform(values: MutableList<Int>) {
    var prev = 0

    for (i in values.indices) {
        val original = values[i]

        values[i] = abs(values[i] - prev)

        prev = original
    }
}

```

```

}

//
=====
===
// Главная функция
//
=====
===
fun main() {

    var root: Node? = null

    println("=== AVL Tree — Variant 11 ===")

    while (true) {

        println()
        println("--- Menu ---")
        println("1. Add element")
        println("2. Delete element")
        println("3. Search element")
        println("4. Print tree (visual)")
        println("5. Print in-order traversal")
        println("6. Execute variant 11 task")
        println("0. Exit")
        print("Your choice: ")

        val choice = readln().toInt()

        when (choice) {

            // Добавить элемент
            1 -> {
                print("Enter value to add: ")
                val value = readln().toInt()

                root = insert(root, value)

                println("Element $value added.")
            }

```

// Удалить элемент

```
2 -> {  
    print("Enter value to delete: ")  
    val value = readln().toInt()  
  
    root = removeNode(root, value)  
  
    println("Element $value deleted (if it existed).")  
}
```

// Поиск элемента

```
3 -> {  
    print("Enter value to search: ")  
    val value = readln().toInt()  
  
    val (found, steps) = search(root, value)  
  
    if (found) {  
        println("Found! Steps taken: $steps")  
    } else {  
        println("Not found. Steps taken: $steps")  
    }  
}
```

// Печать дерева

```
4 -> {  
    println()  
    println("Tree (right branch on top):")  
  
    if (root == null) {  
        println("(empty)")  
    } else {  
        printTree(root)  
    }  
}
```

// Симметричный обход

```
5 -> {  
    if (root == null) {  
        println("Tree is empty.")  
    } else {  
        printInOrder(root)  
    }  
}
```

```
}  
}
```

```
// Задание варианта 11
```

```
6 -> {
```

```
    if (root == null) {  
        println("Tree is empty! Add elements first.")  
        continue  
    }
```

```
    println()  
    println("===== Variant 11 task =====")
```

```
    // Исходное дерево  
    println()  
    println("[0] Initial tree:")
```

```
    printTree(root)  
    printInOrder(root)
```

```
    // ---- Шаг 1 ----  
    val values1 = mutableListOf<Int>()
```

```
    inOrder(root, values1)
```

```
    step1Transform(values1)
```

```
    println()  
    println("[1] After transformations:")
```

```
    print("Values: ")
```

```
    for (v in values1) {  
        print("$v ")  
    }
```

```
    println()
```

```
    root = rebuildFromVector(values1)
```

```
    println()
```

```

println("Rebuilt tree after step 1:")

printTree(root)
printInOrder(root)

// ---- IIIar 2 ----
val values2 = mutableListOf<Int>()

inOrder(root, values2)

println()
println("[2] Applying  $ki = |ki - ki-1|$ ")

print("Before: ")

for (v in values2) {
    print("$v ")
}

println()

step2DifferenceTransform(values2)

print("After: ")

for (v in values2) {
    print("$v ")
}

println()

root = rebuildFromVector(values2)

println()
println("[Final] Tree after all transformations:")

printTree(root)
printInOrder(root)

println("=====")
}

```

```
// Выход
0 -> {
    println("Goodbye!")
    return
}

else -> {
    println("Invalid choice. Please try again.")
}
}
}
```

4 Изображения с демонстрацией работы программы

Запустить программу. Используя пункт меню "1. Add element", добавить 10 элементов, включая отрицательные и кратные 4: -5, -3, 8, 12, -4, 7, 16, 20, 2, -8.

Все вставки выполняются с автоматической балансировкой — дерево остаётся AVL-деревом после каждой операции

```
--- Menu ---
1. Add element
2. Delete element
3. Search element
4. Print tree (visual)
5. Print in-order traversal
6. Execute variant 11 task
0. Exit
Your choice: 4

Tree (right branch on top, each level = 4 spaces):
      20
     16
    12
   8
  7
 2
-3
  -4
  -5
  -8
```

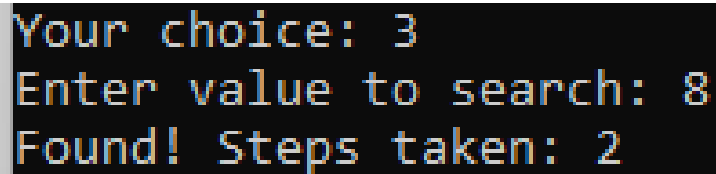
Рисунок 1 – Добавление элементов и визуальный вывод дерева

Дерево выводится в консоль в повёрнутом виде: правая ветвь — сверху, левая — снизу, каждый уровень сдвинут на 4 пробела вправо. По расположению узлов можно визуально убедиться в сбалансированности: дерево не вырождается в список, ветви примерно одинаковой длины

```
Your choice: 5
In-order traversal: -8 -5 -4 -3 2 7 8 12 16 20
```

Рисунок 2 – Вывод в возрастающем порядке

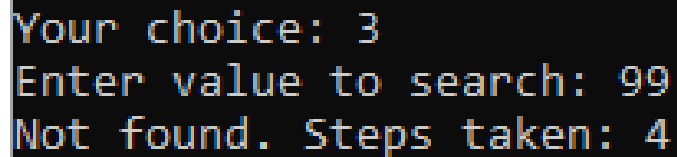
Затем выбрать пункт "5. Print in-order traversal". Программа выводит все значения в возрастающем порядке — это подтверждает, что структура является корректным деревом поиска



```
Your choice: 3
Enter value to search: 8
Found! Steps taken: 2
```

Рисунок 3 – Поиск существующего элемента

Поиск существующего элемента (например, 8). Программа нашла элемент и показала, через сколько узлов прошёл алгоритм. Благодаря балансировке количество шагов не превышает высоту дерева ($\log_2(10) \approx 3-4$).



```
Your choice: 3
Enter value to search: 99
Not found. Steps taken: 4
```

Рисунок 4 – Поиск несуществующего элемента

Поиск несуществующего элемента (например, 99). Алгоритм дошёл до листа и не нашёл элемент, вернув количество пройденных вершин.


```
Your choice: 2
Enter value to delete: 12
Element 12 deleted (if it existed).

--- Menu ---
1. Add element
2. Delete element
3. Search element
4. Print tree (visual)
5. Print in-order traversal
6. Execute variant 11 task
0. Exit
Your choice: 4

Tree (right branch on top, each level = 4 spaces):
      20
     16
    8
   7
  2
-3
  -4
-5
  -8
```

Рисунок 5 – Удаление элемента

Выбрать пункт "2. Delete element", ввести значение 12. Программа выводит: "Element 12 deleted (if it existed)." После этого вывести дерево пунктом 4. На скриншоте видно, что дерево перестроилось: узел 12 исчез, а соседние узлы сдвинулись так, чтобы сохранить AVL-свойство. Если удаление нарушило баланс, автоматически выполнен поворот.

```

Your choice: 6

===== Variant 11 task =====

[0] Initial tree:
      20
     /  \
    16   8
   /  \  /  \
  7   2 /  \  \
 /  \ /  \  \
-3 -4 -5 -8
In-order traversal: -8 -5 -4 -3 2 7 8 16 20

[1] After replacing negatives with abs() and halving multiples of 4:
    Values: 4 5 2 3 2 7 4 8 10

    Rebuilt tree after step 1:
          10
         /  \
        8   7
       /  \ /  \
      5   3 4   2
     /  \
    2   3
In-order traversal: 2 3 4 5 7 8 10

[2] Applying  $k_i = |k_i - k_{i-1}|$  (in-order,  $k_{prev}=0$  for first):
    Before: 2 3 4 5 7 8 10
    After:  2 1 1 1 2 1 2

[Final] Tree after all transformations:
      2
     / \
    1   1
In-order traversal: 1 2

```

Рисунок 6 – Выполнение задания по варианту

Выбрать пункт "6. Execute variant 11 task". В блоке [0] выводится исходное дерево и его симметричный обход, например: -8 -5 -4 -3 2 7 8 16 20

В блоке [1] отображается результат двух преобразований:

1. Замена отрицательных на абсолютные значения: $-8 \rightarrow 8$, $-5 \rightarrow 5$, $-4 \rightarrow 4$, $-3 \rightarrow 3$
2. Уменьшение в 2 раза кратных 4: $8 \rightarrow 4$, $4 \rightarrow 2$, $8 \rightarrow 4$, $16 \rightarrow 8$, $20 \rightarrow 10$
Результат: 4 5 2 3 2 7 4 8 10

3. Далее выводится перестроенное дерево после шага 1 — визуально оно уже содержит только неотрицательные значения.

В блоке [2] выводятся значения до и после применения $k_i = |k_i - k_{i-1}|$.

Пример для in-order последовательности 2 3 4 5 7 8 10:

$$- k_0 = |2 - 0| = 2$$

$$- k_1 = |3 - 2| = 1$$

$$- k_2 = |4 - 3| = 1$$

$$- k_3 = |5 - 4| = 1$$

$$- k_4 = |7 - 5| = 2$$

$$- k_5 = |8 - 7| = 1$$

$$k_6 = |10 - 8| = 2$$

Before: 2 3 4 5 7 8 10

After: 2 1 1 1 2 1 2

В блоке [Final] выводится итоговое дерево, перестроенное из полученных значений, и его симметричный обход:

In-order traversal: 1 2 10 (значения 1 и 2 встречались несколько раз — дубликаты не хранятся в дереве поиска, остаются уникальные ключи)

5 Вывод

В ходе выполнения лабораторной работы было изучено АВЛ-дерево — разновидность бинарного дерева поиска, которое автоматически поддерживает сбалансированность: разница высот левого и правого поддеревьев в любом узле не превышает 1. Это гарантирует логарифмическую временную сложность $O(\log n)$ для операций вставки, удаления и поиска. Были реализованы следующие алгоритмы:

1. Вставка элемента. После стандартной вставки в бинарное дерево поиска выполняется обратный проход от вставленного узла к корню с пересчётом высот и при необходимости — одиночным или двойным поворотом. Вставка требует не более одного поворота.
2. Удаление элемента. Удалённый узел заменяется минимальным элементом правого поддерева. Балансировка может потребоваться для всех предков удалённого узла — до $O(\log n)$ поворотов.
3. Поиск элемента. Классический поиск в бинарном дереве поиска за $O(\log n)$ шагов благодаря сбалансированности дерева. Подсчёт и вывод количества пройденных вершин позволяет наглядно убедиться в эффективности поиска.
4. Задание варианта 11. Реализовано в три этапа:
 - сбор всех значений через симметричный обход в массив;
 - применение преобразований (abs , деление на 2);
 - вычисление разностей $k_i = |k_i - k_{i-1}|$ по симметричному порядку;
 - перестройка АВЛ-дерева из полученного массива.

После преобразования $k_i = |k_i - k_{i-1}|$ возможно появление одинаковых значений — они хранятся как одна вершина, поскольку АВЛ-дерево поиска хранит уникальные ключи. Практическая ценность АВЛ-деревьев состоит в их предсказуемой производительности независимо от порядка добавления элементов, что отличает их от обычных бинарных деревьев, которые могут вырождаться в список.